

AI/ML Architect

От бизнес-задачи до работающего
AI-сервиса

Для Сурена Акопяна
По вакансии «Интерика Лаб»

18

модулей

90

занятий

400 ч

основной работы

Python и backend · LLM и RAG · OCR и CRM
Архитектура · Надёжность · Команда · Реальный пилот

Содержание

Кликабельные разделы и закладки PDF ведут к нужному этапу.

Начало маршрута

Ориентация и подготовка / 03

Карта 18 модулей / 04

Основные модули

01. Бизнес-задача, архитектура и backend / 10

02. Очередь, CI и первый развёрнутый стенд / 11

03. ML, математика и базовая модель / 12

04. Как работает LLM и её API / 13

05. JSON, валидация и вызов инструментов / 14

06. Первая CRM-интеграция и полный сценарий / 15

07. Первый RAG: документы и источники / 16

08. Качество поиска и vector DB / 17

09. Оценка RAG и LLM-систем / 18

10. OCR и извлечение документов / 19

11. Агенты и управляемые workflows / 20

12. Надёжность, конкурентные изменения и outbox / 21

13. Мониторинг, SLO и стоимость / 22

14. Безопасность AI и права доступа / 23

15. Нагрузка, выпуск, откат и восстановление / 24

16. Архитектурный разбор и работа через команду / 25

17. Проведение ограниченного пилота / 26

18. Портфолио, защита и подготовка к интервью / 27

Приложения и материалы

Метрики качества и пользы / 28

Пилот, испытания, рабочие шаблоны и интервью / 29

Дополнительные технологии / 37

Видео и курсы / 42

Собеседование за 14 дней / 44

Словарь / 45

Личный трекер / 46

Как пройти маршрут

План под вакансию «Интерика Лаб»; исходный опыт не предполагается.

На какой результат рассчитан план

Цель - самостоятельно спроектировать, реализовать и защитить AI-сервис для бизнес-процесса: Python/backend, LLM, RAG, OCR, CRM, надёжность и организация работы. Сквозной проект - помощник менеджера, который отвечает по разрешённым документам и готовит проверяемые действия в CRM.

Маршрут развивает техническую базу и даёт проверяемые артефакты. Требования вакансии к более чем шести годам опыта и техническому лидерству подтверждаются реальной работой. Учебный проект и проведённый пилот указываются как разные виды опыта.

Выбор старта

- Уверенный backend и опыт эксплуатации: выполнить входную проверку, пропустить только подтверждённые базовые темы; основной упор на AI, измерения и пилот.
- Python знаком, сервисы пока учебные: пройти недостающую базу и модули 1-2 полностью, затем двигаться по результатам проверок.
- Программирование с нуля: сначала отдельный этап Python, SQL, HTTP, Git и самостоятельных проектов. Оценка часов основного маршрута к этому этапу не относится.

Объём и календарь

18 модулей: 400 часов запланированной работы и до 152 часов резерва, всего 400-552 часа. Часы включают изучение, реализацию, проверку и запись результата. Входная диагностика 6-8 часов и подготовка 56-100 часов идут дополнительно, если нужны.

При 20 часах в неделю основной маршрут занимает примерно 20-28 учебных недель; при 10 часах - 40-56; при 30 часах - 14-19. Это расчёт объёма, а не обещание трудоустройства. Ожидание доступов, напарника и потока документов может увеличить календарный срок.

Модуль 17 содержит 24 часа работы и до 16 часов резерва, распределённых на 2-4 календарные недели пилота. Недостаток данных требует продления наблюдения. При непройденном критерии модуль продолжается даже после исчерпания резерва.

Карта основного маршрута

400 часов занятий + до 152 часов резерва. Номер модуля не равен календарной неделе.

№	Модуль	Часы	Доказательство
1	Бизнес-задача, архитектура и backend	24 + 8	API + схема
2	Очередь, CI и первый развёрнутый стенд	20 + 8	Стенд + CI
3	ML, математика и базовая модель	16 + 8	Baseline
4	Как работает LLM и её API	20 + 8	Адаптер + eval
5	JSON, валидация и вызов инструментов	20 + 8	Схемы + проверки
6	Первая CRM-интеграция и полный сценарий	24 + 8	Полный процесс
7	Первый RAG: документы и источники	24 + 8	RAG + источники
8	Качество поиска и vector DB	24 + 8	Сравнение поиска
9	Оценка RAG и LLM-систем	20 + 8	Набор + runner
10	OCR и извлечение документов	28 + 8	Поля + review
11	Агенты и управляемые workflows	24 + 8	Граф + состояния
12	Надёжность, конкурентные изменения и outbox	24 + 8	Outbox + конфликты
13	Мониторинг, SLO и стоимость	20 + 8	SLI + стоимость
14	Безопасность AI и права доступа	24 + 8	Права + угрозы
15	Нагрузка, выпуск, откат и восстановление	24 + 8	Нагрузка + restore
16	Архитектурный разбор и работа через команду	20 + 8	Задача + review
17	Проведение ограниченного пилота	24 + 16	Результаты пилота
18	Портфолио, защита и подготовка к интервью	20 + 8	Портфолио + защита

Входная диагностика

6-8 часов самостоятельной работы; результат определяет старт.

Сделай небольшой сервис

- POST /documents принимает файл, GET /jobs/{id} показывает статус, GET /documents/{id} возвращает метаданные. Ограничь размер и тип загрузки, опиши ошибки.
- Храни клиентов, документы и задания в PostgreSQL. Подними схему миграциями с чистой БД; объясни внешние ключи, уникальность и rollback.
- Создай двух тестовых пользователей разных клиентов. Запрети чтение чужого документа через прямую подстановку ID. Клиента определяй по проверенной учётной записи.
- Добавь Docker Compose, README и проверки успешной загрузки, ошибочного ввода и отсутствия прав. Не добавляй реальные секреты в репозиторий.
- Измени требование: у документа появилась версия и новый обязательный атрибут. Самостоятельно обнови схему, API и проверку; объясни, что будет со старыми данными.

Как оценивать

Для каждого пункта отметь: 0 - не реализовано; 1 - работает только после пошаговой помощи; 2 - самостоятельно работает и можешь объяснить решение и отказ. Это навигация по обучению, а не тест профессионального уровня.

Проблемы с кодом и исключениями ведут в базу A; с HTTP/Git - в B; с данными и транзакциями - в C; с воспроизводимостью и проверками - в D. После повторения снова выполни проваленный сценарий. Любой обход прав требует исправления до работы с чужими данными.

Правило самостоятельности

AI-помощника можно использовать для объяснений, поиска ошибки и обсуждения вариантов. После этого воспроизведи существенную часть без пошаговой подсказки, внеси новое изменение и объясни поведение при отказе. Сохраняй собственные решения и измерения.

Подготовка А-В: Python, Git и HTTP

Пройти только темы, не подтверждённые входной работой.

А. Python для сервиса | 20-40 часов

Коллекции, функции, исключения, классы, модули, type hints, окружение, зависимости, итераторы и context managers. Затем asyncio: I/O, задачи, ограничение параллелизма, тайм-ауты и отмена.

- Напиши CLI для JSON-заявок: проверка обязательных полей, CSV ошибок и тест на испорченный файл.
- Сравни последовательные HTTP-запросы и конкурентные с ограничением одновременности. Корректно обработай тайм-аут и отмену.
- Готовность: меняешь входной формат, объясняешь await, ресурсы и исключения, можешь найти причину ошибки по traceback.

В. Git, Linux и HTTP | 8-16 часов

Командная строка, процессы, пути, окружение и журналы. Git: commit, branch, merge, конфликт, revert. HTTP: метод, URL, headers, body, status, TLS, аутентификация и авторизация.

- Реши учебный merge-конфликт и отмени отдельный commit через revert. Исключи секреты и локальные файлы из репозитория.
- Выполни GET/POST отдельным клиентом. Исследуй 400/401/403/404/500; запусти и корректно останови сервис.
- Готовность: воспроизводишь запрос без интерфейса и восстанавливаешь рабочую версию кода.

Материалы: изучай по задаче занятия

Видео / курс: David Malan / CS50P: курс Python · CS50P: задания к лекциям · Python Tutorial · Python: asyncio: задачи и отмена · Видео / курс: MIT Missing Semester: Version Control (Git) · Pro Git: русское издание · MDN: обзор HTTP

Подготовка C-D: данные, тесты и контейнеры

Подготовительные часы не входят в 400 часов основного маршрута.

C. PostgreSQL и доступ к данным | 16-24 часа

SQL: SELECT, JOIN, GROUP BY, ограничения, связи, индексы и транзакции. Затем SQLAlchemy/Alembic, пул соединений, N+1 и EXPLAIN. Сначала понимание SQL, затем ORM.

- Создай clients, documents, jobs. Найди последние документы клиента, долю ошибок за день и задания без результата.
- Сравни EXPLAIN до и после индекса. Воспроизведи rollback и подъём пустой БД миграциями.
- Готовность: объясняешь инварианты БД и приложения, ограничения уникальности и последствия повторного запроса.

D. pytest и Docker | 12-20 часов

Fixtures, параметризация значимых сценариев, интеграционные проверки и изоляция данных. Dockerfile, Compose, сеть, volumes, конфигурация и логи.

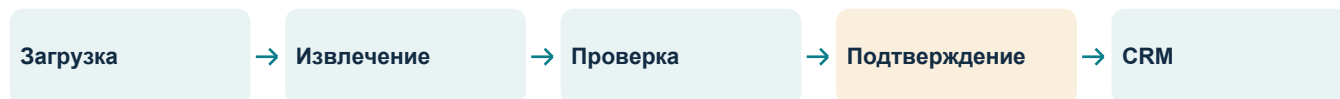
- Подними API и БД с healthcheck. Проверь сохранение данных после пересоздания контейнера и отдельное восстановление из копии на основном маршруте.
- Проверь успешный запрос, невалидный ввод и запрет чужого документа. Используй управляемый fake для внешнего API; реальную интеграцию проверяй отдельно.
- Готовность: чистое окружение воспроизводит запуск, а проверка ловит намеренно внесённую существенную ошибку.

Материалы: изучай по задаче занятия

Видео / курс: Артём Шумейко: SQLAlchemy и Alembic · PostgreSQL: SQL, JOIN и агрегаты · PostgreSQL: транзакции · PostgreSQL: EXPLAIN · SQLAlchemy: Unified Tutorial · Alembic: Tutorial · Видео / курс: TechWorld with Nana: Docker · pytest: fixtures · Docker: Introduction

Сквозной проект: AI-помощник менеджера

Один основной сценарий расширяется постепенно.



Backend проверяет права, состояние и допустимость действия на всём пути.

Два пользовательских пути

Путь 1: менеджер задаёт вопрос по регламентам. Сервис проверяет права, находит разрешённые источники, формирует ответ с привязкой к документу и странице либо сообщает о недостатке сведений.

Путь 2: менеджер загружает счёт. Сервис извлекает текст/поля, проверяет значения и показывает черновик с исходником. Уполномоченный сотрудник подтверждает конкретную версию. Backend выполняет разрешённую операцию CRM и показывает фактический результат.

Первая версия после модуля 6

Один тип текстовых документов, несколько полей, один вид действия и одна тестовая CRM. RAG и сложные сканы добавляются позже. Поддерживай минимальный интерфейс просмотра исходника, исправления полей, подтверждения и статуса; дизайн интерфейса не должен съедать основной объём.

Стек и границы

Python, FastAPI, Pydantic, SQLAlchemy/Alembic, PostgreSQL/pgvector, Celery и один broker, Docker Compose, pytest. Один доступный LLM API либо Ollama; OCR через PaddleOCR. LangGraph появляется для задачи с сохраняемым состоянием. Qdrant - измеряемое сравнение.

Исходники храни отдельно от метаданных и производных векторов. Для учебного стенда допустимо локальное постоянное хранилище, для выбранного развёртывания оцени объектное. Для каждого поля CRM укажи систему-источник.

Подготовь до старта

- Среда разработки и git-репозиторий; тестовая БД/CRM; доступ к модели с лимитом расходов или локальная модель по ресурсам компьютера.
- Открытые, собственные разрешённые, синтетические или корректно обезличенные документы; список типов и происхождения данных.
- Папки docs/architecture, evals, experiments, runbooks; журнал вопросов к владельцу процесса и список непроверенных допущений.

Контрольные точки и учебный ритм

Переход определяется проверкой результата.

После модуля 6: работающий процесс

Стенд запускается, API и worker работают, CI выполняется, пользователь видит исходник и предложение. Запись в тестовую CRM происходит после подтверждения. Проверены ошибки ввода, прав, повтора и потерянного ответа. Fake помечен как непроверенная интеграция.

После модуля 10: измеренное качество

RAG показывает подтверждающие источники, retrieval сравнивается на одном корпусе, OCR оценивается по полям и типам файлов. Есть отдельная финальная выборка, версии компонентов и разбор дорогих ошибок.

После модуля 15: готовность к ограниченному использованию

Воспроизведены отказы и конкурентные изменения, права и удаление проверены, перегрузка управляется. Копия восстановлена в отдельное окружение, откат показан, поддержка может установить состояние операции. Перед пилотом остаются только явно принятые ограничения.

После модуля 18: доказательства компетенций

Диаграммы совпадают с кодом; решения защищаются; другой человек выполнил запуск или review. Есть отчёт реального пилота либо честно обозначенной репетиции, демонстрация и собственные истории решений.

Ритм одного модуля

Пять занятий: вопрос -> материал -> работающий код -> существенная проверка -> запись результата. Начинай с цели занятия; видео и документацию изучай по этой цели. Каждую итерацию заверши демонстрацией и обновлением ADR.

Резерв используй на непройденные критерии. Если времени не хватает, сокращай факультативные технологии и число функций. Основной успешный процесс, права доступа, корректность записей и восстановление остаются критериями готовности.

Модуль 01

Бизнес-задача, архитектура и backend

24 ч занятий + до 8 ч резерва | 5 занятий

Превратить пожелание «нужен AI» в измеримую задачу и работающий HTTP-сервис.

01 / 4 ч. Выбрать один процесс: обработка счёта и подготовка действия CRM. Описать пользователя, исходное время, цену ошибки, допустимые данные, нагрузку и границы автоматизации. Пометить неподтверждённые значения как гипотезы. Нарисовать C4 context и записать первое решение ADR.

02 / 5 ч. Создать FastAPI-приложение: загрузка документа, получение статуса, список документов. Добавить схемы входа/выхода и единый формат ошибок. Проверить HTTP-коды и ограничения размера файла.

03 / 6 ч. Создать таблицы clients, documents, jobs. Добавить миграцию, внешние ключи и транзакцию регистрации документа. Разделить обработчики HTTP, бизнес-логику и обращения к БД.

04 / 5 ч. Разделить HTTP-обработчик, бизнес-логику и хранение. Описать состояния queued, running, needs_review, succeeded, failed. Клиента получать из проверенной учётной записи. Сформировать контракт фонового задания; выполнить переходы синхронно на учебном примере перед подключением worker в модуле 2.

05 / 4 ч. Проверить создание документа, неправильный ввод и чтение чужого ID. Составить README запуска. Объяснить другому человеку путь запроса от API до транзакции.

Критерий готовности

API запускается по инструкции; миграции воспроизводятся; два клиента изолированы; ошибочный ввод имеет понятный ответ. В README есть измеримая цель процесса.

Объясни: Чем аутентификация отличается от проверки прав? Почему `async def` сам по себе не ускоряет OCR?

Сохрани: API, миграции, README, паспорт задачи, схема контекста и первый ADR.

Материалы: изучай по задаче занятия

Видео / курс: Артём Шумейко: курс FastAPI · MDN: обзор HTTP · FastAPI: конкурентность и `async/await` · FastAPI: зависимости · SQLAlchemy: Unified Tutorial · Alembic: Tutorial · pytest: fixtures · Simon Brown: C4 model · Architecture Decision Records

Модуль 02

Очередь, CI и первый развёрнутый стенд

20 ч занятий + до 8 ч резерва | 5 занятий

Получить воспроизводимую основу для долгих AI-операций и регулярной доставки изменений.

01 / 3 ч. Упаковать API, PostgreSQL, Celery и один broker в Docker Compose. Для учебного проекта выбрать Redis либо RabbitMQ и записать причину. Хранить файлы в постоянном томе или объектном хранилище; передавать в задачу ID, а не содержимое большого файла.

02 / 4 ч. Вынести тестовую обработку документа в worker. Хранить статус и результат в прикладной БД. Ограничить время выполнения и число повторов. Показать пользователю статус через GET /jobs/{id}; проверить ошибочный ID и принадлежность задания клиенту.

03 / 5 ч. Ввести уникальный бизнес-ключ задания и проверку повторной доставки. Смоделировать остановку worker. Зафиксировать известный разрыв между записью БД и публикацией события; временно обнаруживать неотправленные задания сверкой, полноценный outbox реализовать в модуле 12.

04 / 5 ч. Добавить CI: значимые тесты API и БД, сборка образа, проверка миграций. Создать структурированные логи с operation_id и tenant_id без содержимого документов и секретов. Отделить настройки разработки от конфигурации стенда.

05 / 3 ч. Развернуть доступный тестовый стенд с аутентификацией и TLS, если он доступен через интернет. Другой человек или чистое окружение должны повторить запуск по README. Провести один успешный запрос, повтор и управляемое падение; сохранить доказательства.

Критерий готовности

Запрос не удерживает HTTP-соединение на время обработки. Статус переживает перезапуск. Повтор в проверенном сценарии не создаёт второй результат. CI проходит, а инструкция воспроизводит стенд.

Объясни: Какие гарантии даёт broker, а какие остаются у приложения? Почему Docker volume ещё не резервная копия?

Сохрани: Развёрнутый каркас, CI, worker, журнал операций и список оставшихся рисков.

Материалы: изучай по задаче занятия

Celery: задачи, retry, acks и limits · Docker: Introduction · GitHub Actions: сборка и тестирование Python

Модуль 03

ML, математика и базовая модель

16 ч занятий + до 8 ч резерва | 5 занятий

Выбирать метрики и понимать, когда задачу уже решают правила или обычный ML.

01 / 2 ч. Разобрать классификацию, регрессию, признаки и целевую переменную. На 30 заявках вручную отработать категории и неоднозначность; для осмысленного сравнения расширить учебный набор доступными размеченными случаями. Зафиксировать состав и ограничения данных, не выдавать маленький пример за оценку production.

02 / 4 ч. Разделить данные на обучение, настройку и финальную проверку. Дубли и варианты одного шаблона держать в одной части. На малом корпусе явно записать ограничения оценки.

03 / 4 ч. Сделать baseline: правила по словам, затем TF-IDF + логистическую регрессию через scikit-learn Pipeline. Обучать преобразования только на тренировочной части данных.

04 / 4 ч. Посчитать precision, recall, F1 и confusion matrix по классам. Разобрать несбалансированный пример, где высокая ассигаса скрывает пропуск важного типа заявки. Изменить порог и сравнить ошибки.

05 / 2 ч. Посчитать скалярное произведение и cosine similarity двух векторов. Объяснить среднее, медиану, p95, вероятность и разброс оценки на малой выборке. Сравнить правила и модель по цене ошибок. Сохранить простой baseline для будущего сравнения с LLM.

Критерий готовности

Есть воспроизводимое сравнение правил и ML, таблица ошибок по классам и перечень ограничений данных. Ты отличаешь улучшение метрики от случайного колебания на маленькой выборке.

Объясни: Почему fit на всех данных создаёт утечку? Когда важнее recall, а когда precision?

Сохрани: Baseline-классификатор + отчёт с метриками и примерами ошибок.

Материалы: изучай по задаче занятия

Видео / кypc: StatQuest: The Confusion Matrix · Видео / кypc: StatQuest: Cross Validation · Видео / кypc: StatQuest: Cosine Similarity · Google: Machine Learning Crash Course · scikit-learn: ошибки и утечка данных · scikit-learn: текстовые признаки и TF-IDF

Модуль 04

Как работает LLM и её API

20 ч занятий + до 8 ч резерва | 5 занятий

Понимать поведение модели и контролировать вызовы, задержку и расход токенов.

01 / 3 ч. Нарисовать путь текста: токены → векторы → Transformer → следующий токен. Разобрать обучение и inference, ограничения контекста и различие языковой правдоподобности и фактической точности.

02 / 4 ч. Сделать адаптер одного доступного API либо Ollama. Реализовать суммаризацию заявки и классификацию. Конфигурацию модели и секреты вынести из кода; возвращать структурированный результат вызова.

03 / 5 ч. Подготовить 30 фиксированных примеров и простой eval-скрипт с ожидаемыми категориями или рубрикой. Сравнить краткую инструкцию, критерии и few-shot. Версионировать промпты и сохранять результат каждого примера; менять один фактор за эксперимент.

04 / 5 ч. Добавить тайм-аут, обработку ограничения частоты запросов, обрыва сети и недоступной модели. Попробовать streaming; отдельно измерить первый токен и полное завершение ответа.

05 / 3 ч. Сохранить модель, настройки, токены, задержку и оценку ответа. Включить маленький регрессионный набор в процесс выпуска. Для локального запуска записать конфигурацию и фактически проверенные возможности. Настроить лимит расходов и правило остановки эксперимента.

Критерий готовности

Эксперимент повторяется на том же наборе; ошибка API не маскируется под успешный ответ. По одному вызову видно, что было отправлено, какая версия использована и сколько он занял.

Объясни: Почему меньшая температура не гарантирует истинность? Что произойдёт при переполнении контекста?

Сохрани: LLM-адаптер + журнал экспериментов по 30 примерам.

Материалы: изучай по задаче занятия

Видео / курс: 3Blue1Brown: But what is a neural network? · Видео / курс: 3Blue1Brown: Transformers, the tech behind LLMs · Видео / курс: 3Blue1Brown: Attention in transformers · Hugging Face: LLM Course, введение · Anthropic: prompt engineering · Anthropic: streaming · Anthropic: подсчёт токенов · Ollama: Chat API

Модуль 05

JSON, валидация и вызов инструментов

20 ч занятий + до 8 ч резерва | 5 занятий

Отделять предложение модели от проверенных данных и разрешённого действия.

01 / 3 ч. Описать схему счёта: поставщик, номер, дата, валюта, сумма, позиции и источник. Различать обязательное поле и допускаемое null. Неизвестное значение сохранять как неизвестное.

02 / 4 ч. Получить structured output через выбранного провайдера. Проверить поддержку конкретной моделью. Добавить обработку отказа, обрезанного ответа и ошибки схемы; ограничить число повторов.

03 / 5 ч. Реализовать проверки сумм, формата даты, валюты и непротиворечивости полей. Для денежных вычислений использовать десятичную арифметику. Сохранить фрагмент исходного документа для проверки человеком.

04 / 5 ч. Описать инструмент поиска клиента в CRM. Backend проверяет пользователя, права и аргументы, сам выполняет вызов и возвращает результат модели. Сначала ограничиться чтением и подготовкой черновика.

05 / 3 ч. Прогнать отсутствующие и лишние поля, неверную сумму, отказ модели и запрещённый вызов. Ввести valid, needs_review, rejected. Сохранять неизменяемую версию предложения с источником. Проверить: правильная JSON-схема с неверными реквизитами не разрешает бизнес-действие.

Критерий готовности

Синтаксис, бизнес-правила и полномочия проверяются отдельно. Ошибочные реквизиты блокируют автоматическую запись. Инструменты не позволяют выполнить произвольную команду.

Объясни: Может ли корректный JSON содержать ложь? Где находится окончательное решение о разрешении действия?

Сохрани: Контракт извлечения + валидаторы + набор ошибочных примеров.

Материалы: изучай по задаче занятия

Видео / курс: Jason Liu: Pydantic is all you need · Видео / курс: Instructor: Structured LLM Outputs · Pydantic: Models · Pydantic: Validators · Anthropic: structured outputs · Anthropic: tool use · Ollama: Structured outputs · Ollama: Tool calling

Модуль 06

Первая CRM-интеграция и полный сценарий

24 ч занятий + до 8 ч резерва | 5 занятий

Довести один документ от загрузки и извлечения полей до подтверждённого действия в тестовой CRM.

01 / 4 ч. Выбрать одну CRM и получить тестовый контур. Описать контакт, компанию, сделку, внешний ID и источник истины для каждого поля. Начать с одного типа документа с текстовым слоем. При отсутствии доступа использовать fake того же контракта и явно отметить непроверенную интеграцию.

02 / 5 ч. Прочитать официальную схему доступа: входящий webhook либо OAuth. Для OAuth изучить обмен кода и обновление токенов, score и ошибки доступа. Секреты хранить отдельно от логов и репозитория.

03 / 6 ч. Реализовать чтение карточки, поиск по внешнему ID и подготовку обновления. Обработать пагинацию, ограничения частоты, неверное поле и удалённый объект. Описать контракт своего CRM-адаптера.

04 / 5 ч. Принять событие CRM, проверить его по правилам поставщика и быстро сохранить для обработки. Ввести журнал событий и ключ дедупликации. Имитировать повтор и изменение порядка уведомлений.

05 / 4 ч. Соединить загрузку, извлечение модуля 5, проверку и подтверждение с записью в CRM. Сохранить версию подтверждённых аргументов. Смоделировать потерянный ответ после успешной записи и выполнить сверку. Показать весь процесс потенциальному пользователю и записать три замечания.

Критерий готовности

Повтор события не создаёт дубликат в проверенных сценариях. Потерянный ответ приводит к сверке состояния. Права ограничены необходимыми операциями, а ошибки видны в журнале.

Объясни: Чем входящий webhook отличается от уведомления о событии? Как восстановиться после неопределённого результата записи?

Сохрани: Работаящая вертикаль, CRM-адаптер, обработчик событий и обратная связь пользователя. Для fake - перечень непроверенных контрактов.

Материалы: изучай по задаче занятия

Видео / курс: Bitrix24: REST API: Incoming Webhooks · Видео / курс: Bitrix24: REST API Events · Битрикс24: первый REST API запрос · Битрикс24: события REST API · amoCRM: OAuth по шагам

Модуль 07

Первый RAG: документы и источники

24 ч занятий + до 8 ч резерва | 5 занятий

Построить поиск по своим документам и ответ с проверяемыми ссылками.

01 / 4 ч. Подобрать 50 небольших документов: регламенты, инструкции, карточки товаров. Начать с Markdown и PDF с текстовым слоем. Подготовить вопросы, на которые корпус отвечает и не отвечает.

02 / 5 ч. Сделать загрузку, хеш файла, версию и извлечение текста. Сохранять исходник отдельно от метаданных и векторов. Разделить текст на фрагменты с `document_id`, страницей, заголовком и клиентом. Обработать повторную загрузку, повреждённый файл и изменение версии.

03 / 6 ч. Получить embeddings и сохранить их в PostgreSQL с pgvector. Реализовать top-k поиск с фильтром прав до передачи фрагментов модели. Проверить размерность и одинаковую модель для индекса и запроса.

04 / 5 ч. Собрать контекст и запрос к LLM. Вернуть текст ответа, использованные фрагменты и ссылки на страницы. При недостатке сведений сообщать об этом, сохраняя полезный вопрос для уточнения.

05 / 4 ч. Сравнить два размера фрагментов на 20 вопросах. Проверить таблицу, длинный раздел, похожие документы и удалённый источник. Отдельно записать ошибки поиска и ошибки генерации.

Критерий готовности

Источники открываются и подтверждают утверждения. Повторная загрузка не размножает документы; чужие фрагменты не попадают в контекст. Есть первая таблица качества поиска.

Объясни: Чем RAG отличается от fine-tuning? Почему наличие ссылки ещё не доказывает верность ответа?

Сохрани: RAG endpoint + загрузчик + корпус и 20 проверочных вопросов.

Материалы: изучай по задаче занятия

Видео / курс: Lance Martin / LangChain: RAG From Scratch · Видео / курс: LangChain: код и видео RAG From Scratch · pgvector: документация · ruypdf: извлечение текста PDF

Модуль 08

Качество поиска и vector DB

24 ч занятий + до 8 ч резерва | 5 занятий

Улучшать retrieval измерениями и выбирать хранилище по ограничениям проекта.

01 / 4 ч. Разобрать dense-поиск, лексический поиск и BM25. Составить сложные запросы с артикулами, сокращениями, опечатками и синонимами. Записать, почему один поиск может не находить точный код товара.

02 / 5 ч. Добавить лексическую ветку и объединение выдач через RRF. PostgreSQL FTS можно взять как отдельный baseline; не называть его ранжирование BM25. Сравнить dense, lexical и hybrid на одном наборе.

03 / 6 ч. Попробовать reranker: получить кандидатов, затем переупорядочить их и сократить контекст. Измерить прирост качества, дополнительную задержку и вычислительные затраты. Не менять одновременно embedding-модель.

04 / 5 ч. Перенести небольшой корпус в Qdrant. Проверить payload-фильтры и индексы. Разобрать точный поиск, ANN, HNSW и компромисс recall/скорость/память; сравнивать на одинаковых векторах и запросах.

05 / 4 ч. Изменить документ, переиндексировать версию и проверить удаление, отзыв доступа и переключение индекса. Описать жизненный цикл исходника, текста, embeddings и кэша. Подготовить ADR pgvector или Qdrant с измерениями и условием пересмотра.

Критерий готовности

В отчёте видны качество и p95 для сравнимых условий. Есть воспроизводимая переиндексация. Ты объясняешь выбор хранилища, не опираясь только на популярность.

Объясни: Почему reranker не восстановит документ, не попавший в кандидаты? Как фильтры влияют на ANN-выдачу?

Сохрани: Сравнение вариантов retrieval + ADR о выборе vector DB.

Материалы: изучай по задаче занятия

Видео / курс: Qdrant: How to Build the Ultimate Hybrid Search · Видео / курс: Lance Martin / LangChain: RAG From Scratch · Видео / курс: Qdrant: Hybrid Search Demo · Qdrant: Filtering · Qdrant: Indexing · pgvector: документация · PostgreSQL: полнотекстовый поиск

Модуль 09

Оценка RAG и LLM-систем

20 ч занятий + до 8 ч резерва | 5 занятий

Заменить оценку «ответ выглядит хорошо» повторяемой проверкой качества.

01 / 3 ч. Собрать 100 вопросов: прямые, неоднозначные, без ответа, по таблицам, с ограниченным доступом. Для каждого записать ожидаемый результат, допустимые источники и тип задачи.

02 / 4 ч. Разделить 70 примеров для разработки и 30 отложенных. Сохранять связанные документы и близкие формулировки в одной группе. После просмотра holdout для настройки подготовить новый финальный набор.

03 / 5 ч. Оценить retrieval отдельно: Hit@k и Recall@k на ответимых вопросах с непустым эталоном. Вопросы без источника оценивать отдельно по корректному отказу. Для ответа проверить правильность, подтверждение источниками и полноту. Вручную сверить расчёт на пяти примерах.

04 / 5 ч. Запустить два варианта через Langfuse либо собственный runner. Для LLM-судьи задать рубрику и сверить с человеком. Важные сценарии выполнить несколько раз из чистого состояния, чтобы увидеть нестабильность. Отдельно проверить фактическую запись в CRM после полного процесса.

05 / 3 ч. Составить таблицу регрессий, задержек, затрат и конечных бизнес-результатов. Зафиксировать версии данных, промпта, retrieval и модели. Добавить проверки при смене модели; ошибки пользователей превращать в новые примеры. Финальную выборку сохранить для честной проверки.

Критерий готовности

Одна команда повторяет оценку; видны числитель и знаменатель каждой метрики. Финальный набор не используется для подбора настроек. Субъективные оценки имеют понятную рубрику.

Объясни: Чем Hit@5 отличается от Recall@5? Почему LLM-судья не заменяет эталон и проверку человеком?

Сохрани: Набор оценки + runner + отчёт сравнения двух версий.

Материалы: изучай по задаче занятия

Видео / курс: Langfuse: Evaluations Deep Dive · Видео / курс: Langfuse: LLM-as-a-Judge в экспериментах · Langfuse: Datasets · Langfuse: Experiments via SDK · Langfuse: учебные видео · Anthropic: Demystifying evals for AI agents

Модуль 10

OCR и извлечение документов

28 ч занятий + до 8 ч резерва | 5 занятий

Обрабатывать реальные PDF и сканы, показывая непроверенные реквизиты человеку.

01 / 4 ч. Собрать 30-50 разнородных файлов: текстовый PDF, скан, фото, поворот, таблица, плохой контраст. Разметить ключевые поля вручную; держать несколько новых шаблонов для финальной проверки.

02 / 6 ч. Сначала попытаться извлечь текстовый слой; для сканов включать OCR. Сохранять страницы и координаты блоков. Сравнить результат до и после простой коррекции ориентации изображения.

03 / 7 ч. Исследовать layout и таблицы через PP-StructureV3. Отличать чтение символов от восстановления строк таблицы. Проверить переносы строк, объединённые ячейки и десятичные разделители.

04 / 7 ч. Соединить OCR с контрактом извлечения модуля 5. Сопоставить сумму, валюту, дату и позиции с исходными фрагментами. Сделать минимальный экран: страница документа, извлечённые значения, исправления и подтверждение. Сохранять автора и версию исправления.

05 / 4 ч. Посчитать точность каждого важного поля и долю документов, прошедших все проверки. Отдельно оценить качество по типам файлов. Не считать OCR confidence вероятностью правильности бизнес-документа.

Критерий готовности

Видно, какое поле ошибочно и где его проверить в источнике. Документ с противоречивой суммой получает `needs_review`. Новый шаблон проходит через тот же процесс без ручного исправления кода.

Объясни: Почему хороший OCR может дать неправильную сумму счёта? Когда extraction без OCR предпочтительнее?

Сохрани: Документный pipeline + размеченные файлы + отчёт по полям.

Материалы: изучай по задаче занятия

Видео / курс: PaddleOCR: извлечение текста на Python · pypdf: извлечение текста PDF · PaddleOCR: Quick Start · PaddleOCR: PP-StructureV3 · Pydantic: Validators

Модуль 11

Агенты и управляемые workflows

24 ч занятий + до 8 ч резерва | 5 занятий

Строить многошаговый процесс с сохраняемым состоянием и контролируемыми действиями.

01 / 4 ч. Прочитать Building effective agents. Для своего процесса отдельно выписать детерминированные шаги и места выбора модели. Обосновать, где нужен workflow, а где полезна ограниченная автономия.

02 / 5 ч. Описать граф: загрузка → extraction → validation → review → CRM. Определить состояние, вход и выход каждого узла, ошибки и конечные статусы. Реализовать переходы через LangGraph.

03 / 6 ч. Добавить checkpoint и остановку перед записью. Подтверждать конкретную версию неизменных аргументов, хранить пользователя и время. Перед исполнением повторно проверить полномочия и актуальность значимых данных CRM; при конфликте запросить новое подтверждение по принятому правилу.

04 / 5 ч. Дать модели два инструмента чтения. Ограничить число шагов, время, расходы и доступные операции. Проверить пустой ответ инструмента, бесконечное уточнение и просьбу вызвать неразрешённую функцию.

05 / 4 ч. Перезапустить приложение посреди процесса, возобновить review и проверить повтор узла. Сравнить граф с простой функцией orchestration. Сформулировать инвариант: checkpoint сохраняет прогресс, а корректность внешнего эффекта обеспечивается отдельно. Подготовить сценарий одновременного подтверждения для модуля 12.

Критерий готовности

Процесс восстанавливается из сохранённого состояния; ожидание review видно пользователю. Лимиты останавливают лишние циклы. Checkpoint не считается гарантией однократного внешнего действия.

Объясни: Когда один агент хуже обычного workflow? Что произойдёт, если узел с побочным эффектом выполнится повторно?

Сохрани: Граф процесса + сохранение состояния + подтверждение и возобновление.

Материалы: изучай по задаче занятия

Видео / курс: freeCodeCamp: LangGraph: Complex AI Agents · Anthropic: Building effective agents · LangGraph: Workflows and agents · LangGraph: сохранение состояния · LangGraph: Interrupts · Видео / курс: LangChain Academy: Introduction to LangGraph

Модуль 12

Надёжность, конкурентные изменения и outbox

24 ч занятий + до 8 ч резерва | 5 занятий

Сохранять корректность системы при остановке воркера и сбоях внешних сервисов.

01 / 4 ч. На работающем worker разобрать подтверждение доставки, повтор и отмену. Разделить долгие OCR-задачи и быстрые операции по очередям при измеренной необходимости. Определить конечные состояния и обработку постоянно падающего задания без блокировки остальных.

02 / 5 ч. Разделить временные и постоянные ошибки. Добавить тайм-ауты, ограниченные повторы, exponential backoff и jitter. Ошибки схемы или прав не повторять бесконечно.

03 / 6 ч. Проверить два одновременных подтверждения и изменение карточки CRM после review. Обеспечить атомарный переход состояния в БД, уникальный бизнес-ключ и контроль версии. Согласовать повторное чтение, пересчёт или ручное разрешение конфликта с учётом возможностей CRM.

04 / 5 ч. Реализовать минимальный transactional outbox: изменение данных и событие в одной транзакции, отдельный отправитель. Добавить сверку зависших заданий и CRM-действий с неопределённым результатом.

05 / 4 ч. Остановить worker до и после внешней записи; перезапустить отправитель outbox; потерять ответ CRM. Воспроизвести гонку подтверждений и записать итог БД/CRM. Не обещать универсальное exactly-once: при ограничениях внешнего API неопределённый исход требует сверки и ручного решения.

Критерий готовности

В журнале нет бесследно потерянных заданий в проверенных отказах. Повтор не умножает внешний эффект. Есть конечный failed-статус и управляемый повтор оператором.

Объясни: Почему очередь не обеспечивает exactly-once бизнес-действие? Как закрывается разрыв между записью в БД и публикацией события?

Сохрани: Асинхронная обработка + outbox + отчёт о пяти сценариях сбоя.

Материалы: изучай по задаче занятия

Видео / курс: NeuralNine: Celery, RabbitMQ & Redis · Celery: задачи, retry, acks и limits · Celery: Redis как broker · Chris Richardson: Transactional Outbox · PostgreSQL: изоляция транзакций

Модуль 13

Мониторинг, SLO и стоимость

20 ч занятий + до 8 ч резерва | 5 занятий

Понимать, где система ошибается, сколько стоит успешный сценарий и что улучшать.

01 / 3 ч. Выбрать пользовательские показатели: успешная обработка, ожидание результата, ручная проверка. Разделить SLI - измерение и SLO - согласованную цель. Определить окно измерения и знаменатель.

02 / 4 ч. Передавать `trace_id` через API, очередь, OCR, retrieval, LLM и CRM. Добавить структурированные логи. Не записывать секреты; содержание документов сохранять только по продуманным правилам доступа и хранения.

03 / 5 ч. Измерить ошибки, глубину и возраст очереди, p50/p95, время первого токена и полного ответа. Настроить простую панель и один сигнал о накоплении необработанных заданий.

04 / 5 ч. Посчитать LLM, embeddings, OCR, повторы, инфраструктуру и лицензии за период. Добавить время сотрудника на review, исправления и ручные отказы, а также поддержку. Показать стоимость принятого результата отдельно без разработки и с распределёнными затратами внедрения; цены брать на дату измерения.

05 / 3 ч. Попробовать сокращение контекста или более дешёвый маршрут модели. Повторить eval модуля 9. Кэш ключевать с учётом клиента, прав и версий; проверить отзыв доступа.

Критерий готовности

По одному трассе можно локализовать сбой. Видны качество, скорость и стоимость до/после изменения. Панель HTTP 200 дополнена проверкой правильности бизнес-результата.

Объясни: Почему низкая стоимость одного вызова не равна низкой стоимости заявки? Что показывает p95, чего не видно по среднему?

Сохрани: Трассы, панель SLI/SLO, сигнал об очереди, отчёт качества/стоимости и расчёт с учётом работы людей.

Материалы: изучай по задаче занятия

Видео / курс: OpenTelemetry: демо Grafana/Loki/Prometheus/Tempo · Видео / курс: Google Cloud: Getting started with SLOs · OpenTelemetry: Python Getting Started · Prometheus: типы метрик · Google SRE: Implementing SLOs · Langfuse: учебные видео

Модуль 14

Безопасность AI и права доступа

24 ч занятий + до 8 ч резерва | 5 занятий

Ограничить доступ к данным и действиям независимо от текста, сгенерированного моделью.

01 / 4 ч. Нарисовать границы доверия: пользователь, файл, retrieval, LLM, инструмент, CRM. Для каждой границы выписать проверяемую учётную запись, допустимые данные и разрешённые операции.

02 / 5 ч. Подготовить инструкции-атаки в вопросе, PDF и результате инструмента. Проверить, что они не дают новых полномочий. Вывод модели проверять перед исполнением и отображением в HTML.

03 / 6 ч. Проверить изоляцию в БД, vector search, файлах и кэше. Разделить права менеджера, проверяющего и администратора. Подменить ID, отозвать доступ во время ожидания review, проверить задания и ссылки на файлы. Для RLS учесть роли, обходящие политики.

04 / 5 ч. Ограничить загрузки по размеру, типу и обработке. Для скачивания по URL запретить недопустимые назначения и проверить перенаправления. Использовать минимальные права интеграций и фильтрацию секретов в логах.

05 / 4 ч. Прогнать 20 негативных сценариев: удаление, отзыв прав, запись без подтверждения, атаки из файлов. Описать сроки хранения исходников, производных данных, журналов и резервных копий, разрешённых получателей. Исправить обходы и повторить проверки; записать оставшиеся ограничения.

Критерий готовности

Доступ и действия проверяет backend. Удалённый либо недоступный документ не выдаётся через поиск и кэш в проверенных сценариях. Есть перечень покрытых угроз и оставшихся ограничений.

Объясни: Почему системного промпта недостаточно для авторизации? Какие пути утечки остаются помимо самой БД?

Сохрани: Модель угроз на одну страницу + результаты 20 негативных проверок.

Материалы: изучай по задаче занятия

Видео / курс: Jeff Crume / IBM: OWASP и атаки на LLM · OWASP: Prompt Injection · OWASP: защита от LLM Prompt Injection · OWASP: безопасная загрузка файлов · PostgreSQL: Row Security Policies

Модуль 15

Нагрузка, выпуск, откат и восстановление

24 ч занятий + до 8 ч резерва | 5 занятий

Развернуть воспроизводимый сервис, восстановить данные и откатить неудачное изменение.

01 / 4 ч. Проверить конфигурации стенда, API, worker, хранилища и broker. Обновить CI и разделение данных/секретов. Описать совместимость миграций и незавершённых заданий с версиями приложения, промпта, схемы извлечения и индекса.

02 / 5 ч. Задать обычную нагрузку и всплеск: вопросы, загрузки и фоновые задачи. Измерить пропускную способность, p95, возраст очереди и ресурсы. Указать конфигурацию, mocks и реальные вызовы. Найти узкое место; сверить прогноз модуля 1 с измерением.

03 / 6 ч. Один клиент отправляет большой пакет. Ввести квоты, ограничение параллелизма и контролируемый приём новых задач; измерить влияние на другого клиента. Для длительного отказа LLM ограничить повторы, показать статус и ручной путь. Проверить readiness/liveness и graceful shutdown.

04 / 5 ч. Сделать резервную копию БД, файлов и конфигурации; восстановить в отдельном окружении. Проверить согласованность документов и индекса. Измерить время восстановления и фактическую потерю данных; сопоставить с заранее согласованными RTO и RPO.

05 / 4 ч. Откатить приложение и промпт на предыдущую версию. Разобрать совместимость миграций. Написать инструкции на случаи «LLM недоступна», «очередь растёт», «CRM не отвечает».

Критерий готовности

Другой разработчик запускает сервис по README. Восстановление проверено, а не только запланировано. В нагрузочном отчёте указаны железо, профиль запросов и пределы эксперимента.

Объясни: Чем rollback приложения отличается от отката данных? Почему резервная копия без проверки восстановления недостаточна?

Сохрани: Развёрнутый стенд + CI + runbook + протокол восстановления.

Материалы: изучай по задаче занятия

Видео / курс: TechWorld with Nana: Docker · Видео / курс: Stephen Thorne, Google: Getting Started with SRE · Docker: Introduction · GitHub Actions: сборка и тестирование Python · PostgreSQL: SQL dump и восстановление · Locust: первый нагрузочный сценарий · Google SRE: Handling Overload

Модуль 16

Архитектурный разбор и работа через команду

20 ч занятий + до 8 ч резерва | 5 занятий

Связывать технические решения с ограничениями бизнеса и организовывать работу команды.

01 / 3 ч. Обновить C4 context/container по реально работающей системе. Подписать границы доступа, фоновые пути и источники истины. Попросить другого разработчика проследить успешную операцию, конфликт и восстановление без устных подсказок.

02 / 4 ч. Пересмотреть минимум пять ADR, накопленных с первого модуля: модель/API, vector DB, workflow, очередь, eval. Сопоставить предположения с измерениями. Записать последствия и три условия, при которых архитектуру потребуется изменить.

03 / 5 ч. Оценить задержку, объём хранения и стоимость при трёх профилях нагрузки. Для локальной модели учесть веса, KV cache, параллельные запросы, оборудование и работу команды, а не только отсутствие платы за токены.

04 / 5 ч. Передать другому разработчику самостоятельную небольшую задачу: обработчик webhook, сверку CRM или экран review. Описать контракт, примеры, критерии приёмки и зависимости. Согласовать оценку и способ демонстрации. Если напарника нет, организовать внешний review и явно указать ограничение опыта.

05 / 3 ч. Провести review и приёмку: успешный сценарий плюс существенный отказ. Обсудить одно разногласие и зафиксировать принятое решение. Передать запуск/поддержку по инструкции. Объяснить владельцу процесса цену, ограничения и следующий шаг за три минуты.

Критерий готовности

Диаграммы совпадают с проектом. ADR содержат осмысленные альтернативы и последствия. Задачи можно передать разработчику без устных догадок о желаемом результате.

Объясни: Как отказать в лишней сложности аргументированно? Как изменится решение при десятикратном росте нагрузки?

Сохрани: Актуальные C4/ADR, план итерации, переданная задача, review и подтверждение самостоятельного запуска другим человеком.

Материалы: изучай по задаче занятия

Видео / курс: Simon Brown: архитектура с C4 · Видео / курс: Tanya Reilly: The Staff Engineer's Path · Simon Brown: C4 model · Architecture Decision Records · Google: How to do a code review

Модуль 17

Проведение ограниченного пилота

24 ч занятий + до 16 ч резерва | 5 занятий

Проверить полезность системы на реальном процессе и получить измеренный результат внедрения.

01 / 4 ч. Найти владельца процесса и согласовать участников, один тип документов, одну CRM, разрешённые операции, доступы и поддержку. Описать критерии успеха/остановки и ручной путь. Проверить допустимость данных и выбранного способа работы с моделью. Использовать протокол пилота из приложений.

02 / 5 ч. Измерить исходный ручной процесс на сопоставимых случаях: активное время, ожидание, исправления, ошибки. Запустить теневого режим без записи в CRM. Сравнить предложения с решением сотрудника и устранить критические ошибки перед допуском.

03 / 6 ч. Включить ограниченный поток с обязательным подтверждением. Записывать версию предложения, подтверждающего, итог CRM и время. Ежедневно разбирать обращения и проверять случайную выборку принятых операций. Существенные ошибки добавлять в регрессионный набор.

04 / 5 ч. Проводить поддержку по runbook, измерять расходы и ручную работу. Устойчивость испытывать управляемо на стенде. Передать диагностику другому человеку. При изменении системы повторить связанные eval и фиксировать версии для каждого периода пилота.

05 / 4 ч. Сравнить результат с исходным процессом и согласованными порогами. Подготовить решение: расширить, продлить с конкретными исправлениями или остановить сценарий. Если реальных пользователей нет, выполнить учебную репетицию и оставить пункт «реальный пилот» неподтверждённым.

Критерий готовности

Есть фактические данные пилота, состав выборки, журнал проблем, стоимость с работой людей и решение владельца процесса. Учебная репетиция помечена отдельно; календарный срок не заменяет достаточные данные.

Объясни: Как отличить высвобождённое время от реальной экономии? Что остановит пилот и кто принимает это решение?

Сохрани: Паспорт, исходные измерения, журнал использования/инцидентов, отчёт и решение о следующем этапе.

Материалы: изучай по задаче занятия

Google SRE: Implementing SLOs · Google SRE: Postmortem Culture · Anthropic: Demystifying evals for AI agents

Модуль 18

Портфолио, защита и подготовка к интервью

20 ч занятий + до 8 ч резерва | 5 занятий

Показать завершённый результат и честно объяснить его качество, ограничения и следующий шаг.

01 / 3 ч. Зафиксировать версию системы и прогнать финальный набор. Проверить новые документы и ранее не виденные формулировки. Если набор использовался для настройки, обозначить это и подготовить свежую проверку.

02 / 4 ч. Провести демонстрацию: ответ с источником, вопрос без ответа, ошибка OCR, отказ LLM, повтор CRM-события. Показать ручное подтверждение и путь диагностики по `trace_id`.

03 / 5 ч. Оформить репозиторий: запуск, архитектура, данные, eval, стоимость, безопасность и runbook. Записать видео на 5-10 минут. Для каждого числа указать условия измерения.

04 / 5 ч. Оформить результаты модуля 17: исходный процесс, объём данных, качество, время, расходы, ограничения и решение по пилоту. Для каждого утверждения о внедрении указать подтверждение и личный вклад. При отсутствии реального пилота честно показать учебный эксперимент и план проверки.

05 / 3 ч. Провести 30-минутную защиту: требования, схема, компромиссы, отказ, экономика. Подготовить три истории о решении проблемы по схеме контекст → выбор → результат → вывод.

Критерий готовности

Работодатель может увидеть полный процесс, проверить цифры и понять личный вклад. Ты объясняешь компромиссы, отказы и оставшиеся ограничения. Учебный стенд, репетиция и реальный пилот различаются явно.

Объясни: Что ещё надо проверить до расширения пилота? Какой результат покажет, что автоматизация действительно полезна?

Сохрани: Репозиторий, демонстрация 5-10 минут, отчёт пилота/репетиции, три истории о личных решениях и репетиция интервью.

Материалы: изучай по задаче занятия

Видео / курс: Simon Brown: архитектура с C4 · Видео / курс: Tanya Reilly: The Staff Engineer's Path · Google SRE: Postmortem Culture · Google SRE: Implementing SLOs · Google: How to do a code review

Как измерять качество и пользу

Учебные наборы дают предварительную оценку; пороги для пилота согласуются с владельцем процесса.

Правила сравнения

Фиксируй версии и условия, показывай числа вместе с процентами, отдельно разбирай редкие дорогие ошибки. Финальная выборка не используется для подбора настроек; после её использования для настройки нужен новый финальный набор.

Показатель	Как считать и интерпретировать
Hit@k	Доля ответимых вопросов, где среди k кандидатов есть релевантный источник. Вопросы без источника оцениваются отдельно по отказу.
Recall@k	Для вопроса с непустым эталоном: найденные релевантные источники / все размеченные релевантные источники. Затем среднее; нужен достаточно полный эталон.
Правильность ответа	Доля ответов, прошедших заранее описанную рубрику. Спорные случаи разбираются человеком.
Подтверждение источником	Для существенного утверждения проверить, поддерживает ли его указанный фрагмент. Наличие ссылки само по себе недостаточно.
Точность полей	Верные значения / размеченные случаи. Отдельно по критичности полей и типам документов.
Успех операции	Корректно завершённые и принятые бизнес-операции / допустимые операции. Проверяется конечное состояние CRM.
Стабильность	Повторить важные сценарии из чистого состояния. Показать число успешных прогонов и типы расхождений.
Задержка и ожидание	p50/p95 при описанном профиле нагрузки; отдельно время очереди, первый токен и завершение процесса.
Стоимость результата	Все согласованные расходы периода, включая повторы и ручную работу / принятые операции.
Эффект для пользователя	Сравнение активного времени и исправлений на сопоставимых документах до и после внедрения.

Материалы: изучай по задаче занятия

Anthropic: Demystifying evals for AI agents · Google SRE: Implementing SLOs

Приложение А. Подготовка и запуск пилота

Модуль 17: 24 часа занятий + до 16 часов резерва; 2-4 календарные недели

Цель и границы

Проверить, приносит ли система пользу в реальном процессе и может ли команда её поддерживать. Календарная длительность учитывает ожидание документов, обратной связи и повторных наблюдений; она не означает 2-4 недели полной занятости разработчика.

Сценарий: сотрудник загружает документ, система извлекает согласованные поля и предлагает изменение в выделенной CRM. Сотрудник проверяет предложение и подтверждает действие. Для пилота выбираются один тип документов, одна интеграция и небольшая группа пользователей.

Паспорт пилота: заполнить до запуска

Проверь запросы модели, логи и резервные копии. Искусственные идентификаторы только в интерфейсе не исключают передачу исходных данных через другие компоненты.

- Владелец бизнес-процесса, технический ответственный, участники и контакт поддержки.
- Разрешённые данные и операции; документы и действия вне границ пилота.
- Обязательные подтверждения, рабочие часы поддержки и ручной обходной путь.
- Метрики, состав и объём выборки, критерии остановки, дата итогового решения.
- Права доступа, допустимость передачи данных LLM-провайдеру, сроки хранения и порядок удаления.

Неделя 1. Исходные измерения и теневого режим

Измерь существующий ручной процесс на репрезентативных случаях: активное время сотрудника, ожидание, результат, исправления и причину отказа. Разделяй простые, сложные и непригодные документы.

Запусти систему без записи: предложение сравнивается с результатом сотрудника. Зафиксируй версии модели, промпта, схемы данных и приложения. Найденные ошибки преврати в проверочные примеры. Результат этапа: исходные метрики, журнал расхождений и решение о допуске к подтверждаемым операциям.

Неделя 2. Ограниченная группа

Включи действия после проверки человеком, начиная с небольшой доли потока. Сохраняй идентификатор операции, версию просмотренного предложения, подтверждение пользователя и результат интеграции.

Ежедневно разбирай ошибки и обращения. Проверяй случайную выборку принятых операций: отсутствие исправления пользователем ещё не доказывает корректность. Классифицируй причины: документ, извлечение, модель, правила, интеграция или интерфейс.

Приложение А. Итоги и критерии пилота

Решение о расширении принимается по согласованным измерениям

Недели 3-4. Устойчивость и передача

При достаточном качестве расширь объём в согласованных границах. Проведи управляемые испытания отказов на стенде. Проверь восстановление, отзыв доступа и передачу поддержки другому человеку. После существенных изменений повтори соответствующие проверки качества.

Двухнедельный вариант объединяет этапы, но не отменяет исходные измерения и итоговую оценку. При малом потоке результат считается предварительным: календарный срок не заменяет необходимую выборку.

Как читать пороги

Условия в таблице - примеры для обсуждения с владельцем процесса, а не универсальные нормы. Согласуй допустимые ошибки отдельно для каждого критичного поля и действия.

Критическими заранее объяви, например, доступ к чужому клиенту, неверную сумму в подтверждённой записи или повторное создание значимого объекта. Каждый такой случай останавливает соответствующий сценарий до разбора. Ноль обнаруженных случаев не доказывает отсутствия риска.

Решение и отчёт

Возможны три исхода: расширять; продлить с конкретными исправлениями; остановить выбранный сценарий. Зафиксируй объём и состав выборки, версии системы, ограничения, фактические метрики, открытые риски и следующего ответственного.

Если доступа к реальному процессу нет, проведи учебную имитацию по этому протоколу. Назови её учебной: она проверяет подготовку к внедрению, но не подтверждает бизнес-эффект у клиента. В портфолио модуля 18 отдельно обозначь учебные и подтверждённые реальные результаты.

Метрика	Пример согласуемого условия
Активное время сотрудника	Снижение медианы хотя бы на 20% на сопоставимых случаях, с учётом проверки и исправлений.
Критические ошибки	Ни одного наблюдаемого случая; каждый случай требует остановки соответствующего сценария.
Ручные исправления	Не выше согласованной доли, например 10%, с отдельным учётом критичности полей.
Завершение операций	Например, не менее 95% допустимых документов проходят процесс в согласованный срок.
Стоимость результата	В согласованном бюджете, включая работу пользователей и поддержку.
Эксплуатация	Поддержка определяет состояние операции и восстанавливает процесс по инструкции.

Приложение Б. Повторы, конфликты и доставка

Испытания 1-6: корректность значимых действий

Протокол испытания

Для каждого сценария сохрани входные данные, предварительное состояние, точку искусственного сбоя, ожидаемый исход, фактический результат, идентификаторы операций и подтверждающие логи.

Проверяй итоговое состояние CRM и локальной БД. Сообщение интерфейса об успешной записи не доказывает, что бизнес-действие завершилось правильно.

Ограничение гарантии

Для сценариев 4-5 объясни возможности конкретного API. Без поддержки идемпотентности или надёжного поиска результата на стороне CRM нельзя обещать универсальное выполнение ровно один раз. Неопределённый исход должен быть виден и иметь процедуру сверки.

Для сценария 6 проверь не только восстановление отправки, но и возможную повторную доставку после сбоя. Потребитель сообщения должен сохранять корректность при повторе.

Сценарий	Ожидаемый результат
1. Один webhook доставлен несколько раз	Событие распознано по устойчивому идентификатору; повтор не создаёт новую бизнес-операцию.
2. Два сотрудника одновременно подтверждают предложение	Побеждает одно допустимое изменение состояния; второй получает понятный текущий статус.
3. Карточка CRM изменилась после подготовки предложения	Проверяется версия или актуальное состояние; конфликт требует пересчёта либо повторного подтверждения.
4. CRM выполнила запись, но ответ потерян	Операция получает неопределённый статус; система сверяет результат перед повтором и предотвращает дубликат доступными средствами.
5. Worker завершился после внешней записи, до фиксации результата	Повторный запуск начинается с проверки состояния и идентификаторов, а не со слепой повторной записи.
6. БД сохранила задание, но публикация в очередь не произошла	Согласованный механизм доставки, например transactional outbox, позволяет восстановить отправку без потери задания.

Приложение Б. Устойчивость и границы доступа

Испытания 7-12: нагрузка, внешние зависимости и AI

Где проводить проверки

Ошибки провайдера, перегрузку и вредоносные документы воспроизводи на стенде с разрешёнными тестовыми данными. Не устраивай разрушительные проверки в клиентской среде.

Для нагрузки заранее выбери объём, длительность и предел остановки. Сравни задержки других клиентов до испытания и во время него; одной успешной операции недостаточно для вывода об изоляции.

Что считать доказательством

Приложи измерения, трассы и конечные состояния. Для проверки доступа покажи, что запрет сработал до выдачи содержимого и передачи недоступного контекста модели.

Для возврата версии сохрани инструкции, измеренное время восстановления и поведение заданий, созданных до отката. Если испытание проведено только учебно, укажи это в отчёте.

Сценарий	Ожидаемый результат
7. Одно задание постоянно падает	Число попыток ограничено; задание изолировано, причина доступна поддержке; остальные задания продолжают выполняться.
8. LLM отвечает 429, тайм-аут или долго недоступна	Повторы ограничены бюджетом и временем; Retry-After учитывается при наличии; доступны статус и ручной путь.
9. Один клиент загружает большой пакет документов	Квоты и ограничение параллелизма сдерживают нагрузку; измеряется влияние на задержки других клиентов.
10. Документ содержит инструкцию отправить данные наружу	Содержимое не получает полномочий; запрещённый вызов блокируется политикой инструментов и доступов.
11. Пользователь запрашивает документ другого клиента	Доступ запрещён на уровне API и извлечения; содержимое не попадает в ответ, контекст модели или доступные пользователю логи.
12. Выпущена неудачная версия промпта или модели	Проверки или мониторинг выявляют ухудшение; доступен возврат версии с учётом совместимости незавершённых заданий.

Приложение В. ADR и архитектурный разбор

Короткие документы, которые помогают принимать и защищать решения

Шаблон ADR

Название и статус: ADR-007. Создание сделок после подтверждения. Предложено / принято / заменено.

- Контекст: задача, ожидаемый объём, известные ограничения и риски.
- Критерии: что важнее при выборе - корректность, срок, стоимость, задержка или поддержка.
- Варианты: минимум два реалистичных решения, включая простое.
- Решение: выбранный вариант и конкретная причина выбора.
- Последствия: обязанности команды, ограничения, стоимость сопровождения и возможные отказы.
- Проверка: измерения и испытания, подтверждающие пригодность.
- Пересмотр: изменение нагрузки, требований или результатов, при котором решение нужно открыть снова.

Пример аргументации

Решение: подтверждённые операции выполняются фоновым worker. Причина: внешняя CRM отвечает нестабильно, а пользователь должен видеть сохранённый статус и возвращаться к операции позже. Цена решения: необходимо поддерживать очередь, идемпотентность, статусы и сверку неопределённых исходов.

ADR объясняет выбор. Пересказ устройства технологии без альтернатив и последствий для проекта недостаточен.

Повестка архитектурного разбора

К встрече подготовь схему и один конкретный спорный вопрос. Выход встречи: решения, открытые вопросы, владельцы и сроки. После разбора обнови схему и соответствующие ADR.

- Кто пользователь, какую операцию он завершает и как измеряется успех?
- Какие компоненты участвуют, где границы доверия и ответственности?
- Где исходные данные и источник истины? Что является производной копией?
- Как проходят успешная операция, повтор, конфликт и восстановление?
- Какие действия требуют подтверждения? Как контролируются полномочия?
- Как проверяются качество AI, доступность, задержки и стоимость?
- Как меняются версии, выкатывается обновление и выполняется откат?
- Какие ограничения приняты сейчас и что потребует изменения архитектуры?

Приложение В. ТСО и работа с разработчиком

Стоимость результата и проверяемые навыки технического лидерства

Стоимость процесса за выбранный период

Полная стоимость = LLM + OCR + инфраструктура + лицензии + поддержка + работа пользователей + доля затрат на разработку.

Работа пользователей включает проверку, исправление и ручную обработку отказов. Поддержка включает расследование инцидентов, восстановление и регулярное сопровождение.

Стоимость принятого результата = полная стоимость периода / число корректно завершённых и принятых бизнес-операций.

- Включай расходы на неудачные попытки и повторные вызовы.
- Не учитывай в знаменателе технические запросы и незавершённые задания.
- Покажи текущую эксплуатационную стоимость и стоимость с распределёнными затратами внедрения.
- Укажи период распределения, объём, ставки труда и допущения.
- Сравнивай с ручным процессом на сопоставимом составе документов.
- Высвобождённое время показывай отдельно от фактической экономии денег.

Карточка передачи задачи

Заполни: цель, границы задачи, контракты API и событий, примеры входа и выхода, доступы, критерии приёмки, известные риски, инструкции запуска.

Подходящая задача другому разработчику: обработчик webhook, сверка результата CRM или экран подтверждения. В задаче должны быть самостоятельное решение и проверяемый результат.

Review и приёмка

Такой опыт можно обсуждать на интервью как совместную работу. Если review выполнил знакомый разработчик учебного проекта, так и укажи. Если внешнего участника нет, самостоятельная проверка полезна, но не подтверждает опыт управления командой.

- Проверь поведение при ошибке и повторе, авторизацию, миграции и наблюдаемость.
- Оцени проверки значимых сценариев и понятность эксплуатации.
- Отделяй обязательные исправления от предпочтений оформления.
- Попроси показать успешный путь и один существенный отказ.
- Дай другому человеку запустить решение по инструкции.
- Сохрани замечания, ответы и принятые решения.

Приложение Г. Вопросы интервью 1-6

Опорные пункты ответа; подкрепляй их примерами из проекта

1. Как перейти от запроса об AI-ассистенте к архитектуре?

Уточнить пользователя, действие, исходный процесс и измеримый результат. Проверить данные, полномочия, интеграции, объём и цену ошибки. Выбрать ограниченный сценарий, описать ручной резервный путь. Предложить пилот и критерии успеха до выбора модели.

2. Когда использовать код, RAG, дообучение или агента?

Правила и точные вычисления передавать детерминированному коду. RAG рассматривать для поиска актуального контекста. Дообучение - при подтверждённой задаче и подходящих данных после измерения базового решения. Агентный цикл оправдывать необходимостью выбирать последовательность действий и контролировать её стоимость.

3. Как понять, почему RAG отвечает неправильно?

Разделить проверку загрузки, разбиения, поиска, ранжирования и генерации. Проверить, существует ли нужный источник и доступен ли он пользователю. Использовать размеченные вопросы и ожидаемые фрагменты. Отличать отсутствие доказательств от неправильного использования найденного контекста.

4. Как безопасно извлекать данные из документов?

Согласовать схему полей, нормализацию и проверки форматов. Сохранять связь поля с исходным фрагментом. Определить критичные поля и случаи обязательной проверки. Отдельно измерять ошибки OCR, извлечения и бизнес-валидации. Уверенный ответ модели не доказывает корректность.

5. Почему для долгих операций нужна очередь?

Она отделяет приём запроса от обработки, позволяет управлять параллелизмом и восстанавливать задания. Объяснить состояния, подтверждение доставки, сроки, повторы и изоляцию неуспешных заданий. Очередь сама по себе не обеспечивает корректность повторных бизнес-действий.

6. Как предотвратить дублирование записи в CRM?

Назначить операции устойчивый ключ, обеспечить ограничения локальной БД и использовать идемпотентность внешнего API, если она есть. При неопределённом исходе сверять результат. Разобрать падение между внешним действием и локальной фиксацией, назвать ограничения выбранной интеграции.

Приложение Г. Вопросы интервью 7-12

Ответ должен объяснять решение, ограничения и способ проверки

7. Как защититься от prompt injection?

Разделить доверенные инструкции и полученный контент. Ограничить инструменты, аргументы, сетевые назначения и права исполнения. Проверять авторизацию вне модели. Подтверждать значимые изменения и испытывать атаки через документы. Один системный промпт не является достаточным контролем.

8. Как обеспечить изоляцию клиентов?

Передавать подтверждённый контекст клиента через API, фоновые задания и поиск. Контролировать доступ к объектам, индексам, кешам и файлам. Проверять пересечение границ отдельными тестами. Учитывать логи, резервные копии и административные операции.

9. Что происходит при недоступности LLM-провайдера?

Применять ограниченные тайм-ауты и повторы, управлять накоплением очереди и показывать состояние. Предусмотреть ручную обработку. Резервную модель подключать после проверки качества, схемы ответа, стоимости и допустимости передачи данных.

10. Какие показатели нужны в production?

Связать бизнес-метрики с качеством и техническими показателями: принятые операции, исправления, длительность процесса, задержки, ошибки, возраст очереди и стоимость. Согласовать целевые уровни обслуживания и условия оповещения. По trace ID восстанавливать историю конкретной операции.

11. Как выпускать изменения модели и промпта?

Версионировать компоненты, запускать фиксированный набор проверок и анализировать изменения по типам сценариев. Ограничивать первоначальную долю пользователей, наблюдать реальные результаты и иметь способ возврата. Продумать работу заданий, начатых на предыдущей версии.

12. Чем подтверждается техническое лидерство?

Примерами согласования требований, выбора компромиссов, декомпозиции, передачи задач и review. Показать разногласие, принятое решение и результат. На инциденте объяснить восстановление и профилактику. Отделять личный вклад от работы команды, учебный опыт от коммерческого.

Дополнительно: фреймворки

Факультативно: выбирать по проекту и уточнениям работодателя.

Выбирать по задаче после основного маршрута; часы не включены в 400-552 часа.

Django и Flask

12-20 ч на обзор обоих · дополнительные часы вне основной программы

Изучить. Django: модели, migrations, admin, middleware и встроенные возможности. Flask: application factory, Blueprints, конфигурация. Сравнить со своим FastAPI-сервисом по требованиям.

Практика 1. Реализовать чтение и создание документа в одном из фреймворков. В другом прочитать официальный пример и проследить запрос до БД.

Практика 2. Написать короткое решение: какой фреймворк выбрал бы для внутренней админки и какой - для текущего API; указать причины и цену миграции.

Готовность. Ты можешь поддерживать небольшой сервис и объяснить выбор; глубокое владение всеми тремя сразу не требуется.

LangChain и LlamaIndex

12-20 ч · дополнительные часы вне основной программы

Изучить. Модели и адаптеры, loaders, retrievers, node/document abstractions, workflow и инструментирование. Понимать, что скрывается за высокоуровневым вызовом.

Практика 1. Повторить небольшой RAG через один фреймворк. Сохранить тот же corpus, retrieval-параметры и eval-набор.

Практика 2. Сравнить объем кода, наблюдаемость, обработку ошибок и возможность заменить провайдера. Не считать меньше строк доказательством лучшей архитектуры.

Готовность. Ты можешь проследить шаги pipeline и объяснить, какие ограничения добавляет выбранная абстракция.

Материалы: изучай по задаче занятия

Видео / курс: Corey Schafer: Django Web App, Part 1 · Видео / курс: Corey Schafer: Flask Web App, Part 1 · Django: первый проект · Flask: Tutorial · Видео / курс: LangChain: Introduction to LangChain, Python · Видео / курс: LlamaIndex Crash Course: AI Agents in Python · Видео / курс: LangChain: код и видео RAG From Scratch · LlamaIndex: Starter Tutorial · LangGraph: Workflows and agents

Дополнительно: поиск и автоматизация

Факультативно: выбирать по проекту и уточнениям работодателя.

Milvus и сравнение vector DB

8-12 ч · дополнительные часы вне основной программы

Изучить. Коллекция, схема, размерность вектора, индексы, фильтрация, обновление и удаление. Сопоставить возможности с уже проверенными pgvector и Qdrant.

Практика 1. Загрузить один и тот же небольшой корпус и векторы в Milvus. Проверить поиск, фильтр клиента, обновление версии и удаление.

Практика 2. Составить таблицу: качество поиска при одинаковых условиях, сложность эксплуатации, резервное копирование и ограничения пилота.

Готовность. Выбор основан на задаче; маленький тест не выдаётся за доказательство производительности на миллионах документов.

n8n

8-12 ч · дополнительные часы вне основной программы

Изучить. Trigger, данные между узлами, credentials, HTTP Request, ветвление и обработчик ошибки. Граница между визуальным workflow и backend-логикой.

Практика 1. Собрать процесс: тестовое событие → HTTP-запрос к AI-сервису → черновик действия → результат. Секреты хранить через механизм credentials.

Практика 2. Повторить событие и имитировать отказ сервиса. Проверить дедупликацию на стороне backend и понятное уведомление оператору.

Готовность. Ты объясняешь, какая часть остаётся в API и почему; повторный запуск workflow не создаёт лишнюю CRM-запись.

Материалы: изучай по задаче занятия

Видео / курс: Qdrant: How to Build the Ultimate Hybrid Search · Milvus: Quickstart · pgvector: документация · Qdrant: Filtering · Qdrant: Indexing · Видео / курс: n8n: Beginner Course · n8n: Learning Paths · Битрикс24: события REST API

Дополнительно: Kubernetes и Airflow

Факультативно: выбирать по проекту и уточнениям работодателя.

Kubernetes

16-24 ч на учебный стенд · дополнительные часы вне основной программы

Изучить. Deployment, Service, Ingress на уровне назначения, ConfigMap, Secret, resources, probes, rollout и PVC. Понять пределы масштабирующих настроек при внешней LLM и очереди.

Практика 1. Развернуть API и worker, настроить readiness/liveness и limits. Для учебной БД описать способ хранения и восстановления; не переносить её бездумно ради упражнения.

Практика 2. Обновить образ, выполнить rollback, удалить Pod и проследить восстановление. Проверить, что секрет не попал в образ и журнал.

Готовность. Ты можешь показать восстановление приложения и объяснить, при каких требованиях Kubernetes оправдывает эксплуатационную сложность.

Apache Airflow

10-16 ч · дополнительные часы вне основной программы

Изучить. DAG, task, зависимости, расписание, retry, catchup/backfill и контроль обработки данных. Различать периодический pipeline и интерактивный запрос пользователя.

Практика 1. Собрать DAG: забрать обновлённые документы → проверить → разбить на фрагменты → обновить индекс → сформировать отчёт.

Практика 2. Перезапустить один этап, обработать пропущенный период и проверить отсутствие дублей. Сравнить такой запуск с задачей Celery по событию.

Готовность. Переобработка периода воспроизводима; Airflow не используется как замена обычному API без обоснования.

Материалы: изучай по задаче занятия

Видео / курс: TechWorld with Nana: Kubernetes · Kubernetes: Basics · Docker: Introduction · Видео / курс: Timothy Davis: Data pipelines with Airflow 3.0 · Airflow: Architecture Overview

Дополнительно: 1С и МойСклад

Факультативно: выбирать по проекту и уточнениям работодателя.

Для этих интеграций основа - официальные описания API: методы и поведение зависят от продукта и конфигурации. Видео по REST и webhooks из модуля 6 даёт общий механизм; конкретные контракты изучаются по ссылкам ниже.

1С: HTTP-сервисы и OData

12-20 ч на учебный обмен · дополнительные часы вне основной программы

Изучить. Справочники, документы, внешние ID, проведение и источник истины. Различие стандартного OData-доступа и собственного HTTP-сервиса для бизнес-операции.

Практика 1. Описать контракт обмена номенклатурой или статусами документов: поля, ID, ошибки, права, дельта изменений и повтор.

Практика 2. В согласованной тестовой конфигурации прочитать данные и выполнить разрешённую операцию. При отсутствии контура сделать mock и перечислить, что ещё требует проверки с разработчиком 1С.

Готовность. Ты можешь согласовать обмен и объяснить обработку дублей; учебный mock явно отделён от работающего подключения к 1С.

МойСклад: товары, остатки и документы

8-12 ч · дополнительные часы вне основной программы

Изучить. Авторизация, metadata, пагинация, фильтры, ограничения запросов и внешние идентификаторы. Проверить семантику остатков и операций по документации.

Практика 1. Загрузить учебный каталог с пагинацией, связать товары с внешними ID и сохранить время синхронизации.

Практика 2. Повторить обмен, обработать удалённую позицию и временную ошибку. Описать разрешение конфликта, когда данные менялись в двух системах.

Готовность. Повторная синхронизация не дублирует сущности; для каждого поля известна система-источник.

Материалы: изучай по задаче занятия

Видео / курс: Bitrix24: REST API: Incoming Webhooks · 1С: HTTP-сервисы · 1С: стандартный REST-интерфейс · МойСклад: JSON API 1.2

Дополнительно: локальные модели и обучение

Факультативно: выбирать по проекту и уточнениям работодателя.

Ollama и собственное размещение LLM

12-20 ч · дополнительные часы вне основной программы

Изучить. Память весов и KV cache, квантование, длина контекста, параллельность, batching и serving. Оценить условия использования выбранной модели и инфраструктурные ограничения.

Практика 1. Прогнать те же 30 примеров и RAG-набор на локальной модели. Записать конфигурацию, расход памяти, скорость, качество и ограничения.

Практика 2. Сравнить с API с учётом оборудования и эксплуатации. vLLM изучать для отдельного serving-эксперимента, если есть подходящее оборудование и потребность.

Готовность. Есть измеренное сравнение; «локальная» не означает автоматически более дешёвую или более качественную систему.

PyTorch, fine-tuning и LoRA

30-60 ч на первый эксперимент · дополнительные часы вне основной программы

Изучить. Dataset/DataLoader, tensors, autograd, loss, optimizer, train/eval, checkpoint. Затем PEFT/LoRA и разница между изменением поведения модели и подачей актуальных внешних знаний.

Практика 1. Обучить небольшую модель по PyTorch Basics. Сохранить checkpoint, повторить inference и проверить результат на отложенных данных.

Практика 2. До LoRA сформулировать гипотезу и собрать разметку. Сравнить базовую модель, улучшенный prompt и дообучение на одном наборе, включая цену и регрессии.

Готовность. Ты можешь показать воспроизводимый эффект и ограничения. Без данных и убедительного baseline запуск дообучения откладывается.

Материалы: изучай по задаче занятия

Видео / курс: A Deep Dive Into Ollama: tools и structured output · Ollama: Quickstart · Ollama: Chat API · Ollama: Structured outputs · Ollama: Tool calling · vLLM: Quickstart · Видео / курс: PyTorch: Introduction to PyTorch · PyTorch: Learn the Basics · Hugging Face: LLM Course, введение · Hugging Face PEFT: LoRA methods

Видео и курсы: маршрут 1

Проверенные страницы материалов; выбирай темы под практику, а не полный просмотр ради объёма.

David Malan / CS50 | EN

Harvard CS50's Introduction to Programming with Python - Full University Course. Темы: Python, исключения, тесты, файлы, ООП.

[Открыть материал](#)

MIT Missing Semester | EN

Version Control (Git). Темы: Git, ветки, конфликты, история.

[Открыть материал](#)

Артём Шумейко | RU

FastAPI Курс. Темы: FastAPI, Pydantic, БД, авторизация.

[Открыть материал](#)

TechWorld with Nana | EN

Docker Tutorial for Beginners. Темы: Docker, Dockerfile, Compose, volumes.

[Открыть материал](#)

StatQuest | EN

Machine Learning Fundamentals: The Confusion Matrix. Темы: ML, метрики, confusion matrix.

[Открыть материал](#)

3Blue1Brown | EN

Transformers, the tech behind LLMs | Deep Learning Chapter 5. Темы: LLM, токены, transformers.

[Открыть материал](#)

Видео и курсы: маршрут 2

Проверенные страницы материалов; выбирай темы под практику, а не полный просмотр ради объёма.

Jason Liu | EN

Pydantic is all you need: Jason Liu. Темы: JSON, валидация, извлечение.

[Открыть материал](#)

LangChain / Lance Martin | EN

RAG From Scratch. Темы: RAG, indexing, retrieval, generation.

[Открыть материал](#)

Qdrant | EN

Demo: Implementing a Hybrid Search System. Темы: hybrid search, Qdrant, retrieval.

[Открыть материал](#)

Langfuse | EN

Langfuse Academy. Темы: evals, ошибки, datasets, tracing.

[Открыть материал](#)

LangChain Academy | EN

Foundation: Introduction to LangGraph - Python. Темы: workflow, state, memory, human-in-the-loop.

[Открыть материал](#)

NeuralNine | EN

Professional Task Queues in Python with Celery, RabbitMQ & Redis. Темы: Celery, очереди, broker, задачи.

[Открыть материал](#)

Подготовка к собеседованию за 14 дней

Отдельный короткий маршрут для уже отправленного отклика.

Назначение

Это подготовка демонстрации и разговора при имеющейся базе. Она не заменяет основной маршрут и опыт внедрений. Если времени мало, выбери один понятный сценарий и подготовь проверяемые ограничения.

Дни 1-4

- 1-2: входная диагностика, разбор вакансии, вопросы о команде, инфраструктуре и ожидаемом результате первых месяцев.
- 3-4: один процесс и работающий API; схема контекста, путь данных и выбранная модель. Подготовь 10-20 разрешённых документов.

Дни 5-10

- 5-6: RAG с источниками либо извлечение полей - выбери то, что можешь объяснить и проверить.
- 7-8: фиксированные примеры, ошибки и несколько повторных прогонов. Отчёт с числителем, знаменателем, задержкой и расходами.
- 9-10: подтверждение действия, CRM-контракт, дубликат события, отказ модели и запрет чужого документа. Реальное подключение либо явно отмеченный fake.

Дни 11-14

- 11-12: README, архитектурные решения, ограничения, runbook и демонстрация 5-10 минут.
- 13-14: две репетиции защиты и три истории: проблема -> личное решение -> результат -> вывод. Раздели учебный и коммерческий опыт.

Что уточнить у компании

Доля backend/интеграций и ML-обучения; типы и объём документов; доступные модели и ограничения данных; ожидаемая работа с 1С; размер и опыт команды; кто отвечает за поддержку; критерии успеха пилота; что архитектор должен сделать в первые 90 дней.

Словарь основных терминов

Понимай термин через действие в своём проекте.

Термин	Рабочий смысл
Baseline	Простое исходное решение, с которым сравнивают улучшение.
Embedding	Числовое представление текста для сравнения и поиска.
RAG	Поиск внешнего контекста перед формированием ответа.
Retrieval / reranking	Получение кандидатов / повторное ранжирование кандидатов.
Holdout	Отдельные примеры для финальной проверки без подбора настроек.
Eval	Воспроизводимая оценка поведения системы по заданным критериям.
Structured output	Вывод по схеме; значения и полномочия проверяются отдельно.
Workflow / checkpoint	Процесс с переходами / сохранённое состояние для продолжения.
Идемпотентность	Повтор операции сохраняет согласованный бизнес-результат.
Transactional outbox	Изменение данных и событие фиксируются одной транзакцией.
Оптимистическая блокировка	Обновление проходит, только если проверенная версия не изменилась.
Backpressure	Ограничение поступающей работы при нехватке ресурсов обработки.
Tenant	Клиент организации, чьи данные и ресурсы изолируются.
Trace	Связанная история прохождения одной операции через компоненты.
SLI / SLO	Измерение качества сервиса / согласованная цель этого измерения.
RPO / RTO	Допустимая потеря данных / целевое время восстановления.
ADR / C4	Запись причины решения / уровни схем архитектуры.
TCO	Полная стоимость владения, включая эксплуатацию и работу людей.

Трекер: модули 1-9

Впиши дату проверки, ссылку на commit/отчёт и следующий пробел. Готовность подтверждается практикой.

Модуль	Дата / доказательство	Следующий пробел
01. Бизнес-задача, архитектура и backend		
02. Очередь, CI и первый развёрнутый стенд		
03. ML, математика и базовая модель		
04. Как работает LLM и её API		
05. JSON, валидация и вызов инструментов		
06. Первая CRM-интеграция и полный сценарий		
07. Первый RAG: документы и источники		
08. Качество поиска и vector DB		
09. Оценка RAG и LLM-систем		
Статусы: не начато / в работе / требует доработки / проверено. Сохраняй фактические часы; по ним корректируй оставшийся календарь.		

Трекер: модули 10-18

Впиши дату проверки, ссылку на commit/отчёт и следующий пробел. Готовность подтверждается практикой.

Модуль	Дата / доказательство	Следующий пробел
10. OCR и извлечение документов		
11. Агенты и управляемые workflows		
12. Надёжность, конкурентные изменения и outbox		
13. Мониторинг, SLO и стоимость		
14. Безопасность AI и права доступа		
15. Нагрузка, выпуск, откат и восстановление		
16. Архитектурный разбор и работа через команду		
17. Проведение ограниченного пилота		
18. Портфолио, защита и подготовка к интервью		

Статусы: не начато / в работе / требует доработки / проверено. Сохраняй фактические часы; по ним корректируй оставшийся календарь.

Начать и зафиксировать прогресс

Первое действие - проверить базу на маленьком сервисе.

Сегодня

- Выбери обработку одного документа и разрешённый набор из 10 примеров.
- Создай репозиторий и паспорт задачи: пользователь, вход, результат, цена ошибки и ограничения.
- Начни входную диагностику. После неё выбери нужные подготовительные блоки и запланируй первые пять занятий.

Как пользоваться материалами

Сначала смотри цель и практику модуля. Открывай указанные видео/курсы по соответствующей теме; не обязательно проходить длинный плейлист целиком. Официальную документацию используй для конкретного API, настройки и гарантий выбранной версии.

Ссылки исходного плана сохранены по темам. Основные материалы выборочно проверены по доступным первичным страницам; полный просмотр видеокурсов не выполнялся. При недоступном видео используй официальную документацию того же модуля. Версии зависимостей фиксируй в проекте.

Что сохранить к финалу

- Код и инструкция запуска; C4 и минимум пять содержательных ADR; контракты API и CRM.
- Размеченные разрешённые данные, eval-runner, отчёты качества и сравнения вариантов.
- Журнал испытаний, восстановление, откат, трассы и runbook поддержки.
- Переданная задача/review; отчёт пилота или честно обозначенной репетиции; demo и рассказ о личном вкладе.

Происхождение и границы документа

План составлен на основе предоставленной вакансии «Интерика Лаб» и исходного плана для Сурена от 5 сентября 2026 года. Обновление усиливает раннюю доставку, проверку бизнес-результата, конкуренцию, перегрузку, командную работу и проведение пилота.

Часы - планировочная оценка, пороги пилота - предмет согласования. Документ не подтверждает текущие навыки, коммерческий стаж или результат найма. Проверка практикой важнее количества просмотренных материалов.